

Week4_예습과제_이재린

태그

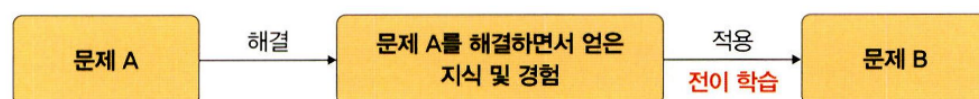
예습과제

CH5. 합성곱 신경망 1

▼ 3. 전이 학습

전이 학습이란?

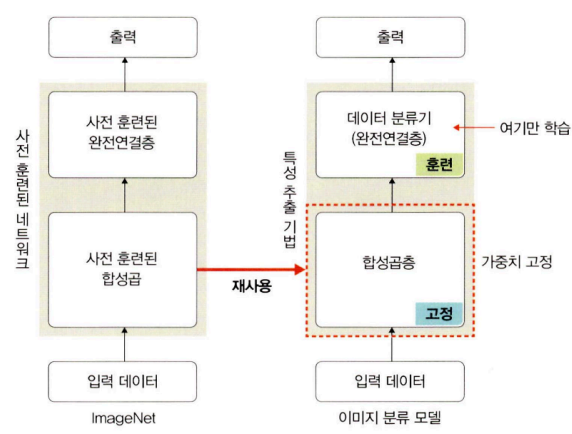
- 전이 학습? CNN 기반 딥러닝 모델에 필요한 많은 양의 데이터 확보의 한계를 해결하는 것. 기존의 큰 데이터셋으로 훈련된 모델의 가중치를 가져와 보정해 사용하는 것
- 네트워크 : 큰 데이터셋으로 사전 훈련된 모델



- 전이 학습의 method : 특성 추출과 미세 조정 기법

특성 추출 기법

- 특성 추출 기법이란? ImageNet 데이터셋으로 사전 훈련된 모델의 마지막 완전연결층 부분만 새로 학습해 만드는 것(나머지는 학습되지 않도록)



- 특성 추출의 구성 for 이미지 분류 : 합성곱층(합성곱층+풀링층) + 데이터 분류기(완전연결층) >> 네트워크의 합성곱층의 가중치를 고정하여 새로운 데이터를 통과시키고, 그 출력을 데이터 분류기에서 훈련시킨다.

ex) Xception, Inception V3, ResNet50, VGG16, VGG19, MobileNet

1. 이미지 데이터에 대한 전처리 방법 정의

```
# 이미지 데이터에 대한 전처리 방법 정의
data_path='/Users/bluecloud/Documents/대학/유런/data/catanddog/train'
transform=transforms.Compose([
    transforms.Resize([256, 256]),
    transforms.RandomResizedCrop(224) ,
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor()
])
train_dataset=torchvision.datasets.ImageFolder(data_path,transform=transform)
train_loader=torch.utils.data.DataLoader(
    train_dataset,
```

```

batch_size=32,
num_workers=8,
shuffle=True
)
print(len(train_dataset))

```

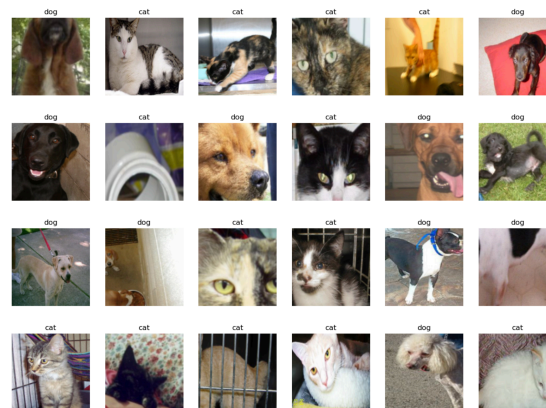
- torchvision.transform : 이미지 데이터를 변환하여 네트워크의 입력으로 사용하도록 변환 >> 이미지는 텐서로 변환됨
- datasets.ImageFolder : 데이터로더가 데이터를 불러올 대상 혹은 경로와 방법(transform)을 정의한다.
- DataLoader : 데이터로더에 train_dataset을 할당하고 불러올 데이터양(batch_size), 데이터를 무작위로 섞을(shuffle) 것인가

2. 학습에 사용될 24개의 이미지 출력

```

#학습에 사용될 24개의 이미지 출력
samples, labels = next(iter(train_loader))
classes = {0:'cat', 1:'dog'} # 개와 고양이에 대한 클래스로 구성
fig = plt.figure(figsize=(16,24))
for i in range(24): #24 개의 이미지 데이터 출력
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()]) # 레이블 정보 ( 클래스 ) 를 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0)))
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)

```



- .next()라는 속성이 없어 next(iter(train_loader))로 대체 >> iter()로 전달된 데이터의 반복자를 꺼내 반환하고 next는 그 반복자가 다음에 출력해야할 요소를 반환한다. >> samples과 labels 값을 train_loader에서 꺼내온다.
- np.transpose는 전치행렬구하기로 행렬의 차원을 바꿔준다. >> 왜? 행렬의 내적 연산 때문에 `torch.Size([32, 3, 224, 224])` >> `torch.Size([224, 224,3])` 과 같은 형태로 변환해서 사용해야한다.

3. 사전훈련된 ResNet18모델 내려받기

```

#사전훈련된 ResNet18모델 내려받기
resnet18=models.resnet18(pretrained=True)

```

- pretrained=True로 사전 학습된 모델을 내려받음을 명시
- ResNet18 모델 : 50개의 계층으로 구성된 합성곱 신경망 >> ImageNet에서 100만개가 넘는 영상으로 신경망 훈련

4. 합성곱층을 고정하여 훈련하지 않도록 고정

```

#합성곱층을 고정하여 훈련하지 않도록 고정
def set_parameter_requires_grad(model, feature_extracting=True):
    if feature_extracting:
        for param in model.parameters():

```

```
param.requires_grad = False
set_parameter_requires_grad(resnet18)
```

5. ResNet18에 완전 연결층을 추가하고 모델의 파라미터 값 확인하기

```
#ResNet18에 완전 연결층을 추가하고 모델의 파라미터 값 확인하기
resnet18.fc=nn.Linear(512,2) # 클래스가 2개

for name,param in resnet18.named_parameters():
    if param.requires_grad:
        print(name,param.data)
```

- model.named_parameters() : 모델에 접근하여 파라미터 값들을 가져온다.
- 파라미터는 weight와 bias가 사용된다.

```
fc.weight tensor([[ 0.0361, -0.0346, -0.0346, ..., -0.0099, -0.0113,  0.0047],
                  [ 0.0118,  0.0404,  0.0116, ...,  0.0028, -0.0277, -0.0214]])
fc.bias tensor([ 0.0367, -0.0225])
```

6. 모델 객체 생성 및 손실 함수 정의

```
#모델 객체 생성 및 손실 함수 정의
model = models.resnet18(pretrained=True) #모델의 객체 생성

for param in model.parameters(): # 모델의 합성곱층 가중치 고정
    param.requires_grad = False

model.fc = torch.nn.Linear(512, 2)
for param in model.fc.parameters(): # 완전연결층은 학습
    param.requires_grad = True

optimizer = torch.optim.Adam(model.fc.parameters())
cost = torch.nn.CrossEntropyLoss() #손실 함수 정의
print (model)
```

7. 모델 학습을 위한 함수 생성

```
def train_model(model,dataloaders,criterion, optimizer,device, num_epochs=13,is_train=True):
    since=time.time() #컴퓨터의 현재 시각을 구하는 함수
    acc_history = []
    loss_history = []
    best_acc = 0.0

    for epoch in range(num_epochs):
        print('Epoch {}/{}'.format(epoch,num_epochs-1))
        print('-'*10)
        running_loss = 0.0
        running_corrects = 0

        for inputs, labels in dataloaders: #데이터로더에 전달된 데이터만큼 반복
            inputs = inputs.to(device)
            labels = labels.to(device)
            model.to(device)
            optimizer.zero_grad() #기울기를 0으로 설정
```

```

        outputs=model(inputs) #순전파
        loss=criterion(outputs,labels)
        _, preds = torch.max(outputs, 1)
        loss.backward() #역전파 학습
        optimizer.step()

        running_loss+=loss.item()*inputs.size(0) #출력 결과와 레이블의 오차를 계산한 결과를 누적하여 저장
        running_corrects+=torch.sum(preds == labels.data) #출력 결과와 레이블이 동일한지 확인한 결과를 누적하여 저장

    epoch_loss=running_loss/len(dataloaders.dataset) #평균 오차 계산
    epoch_acc=running_corrects.double()/len(dataloaders.dataset) #평균 정확도
    print('Loss: {:.4f} Acc: {:.4f}'.format(epoch_loss, epoch_acc))
    if epoch_acc > best_acc:
        best_acc = epoch_acc
    acc_history.append(epoch_acc.item())
    loss_history.append(epoch_loss)
    #모델 재사용을 위한 저장
    torch.save(model.state_dict(),os.path.join('/Users/bluecloud/Documents/대학/유런/data/catanddog','{0:0=2d}.pth'.format(epoch)))
    print()
    time_elapsed= time.time()-since # 실행 시간 ( 학습 시간 ) 을 계산
    print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed//60,time_elapsed%60))
    print('Best Acc: {:.4f}'. format(best_acc))
    return acc_history, loss_history #모델의 정확도와 오차를 반환

```

8. 추가된 완전연결층을 학습하고 얻어지는 파라미터를 옵티마이저에 전달해서 전달

```

#추가된 완전연결층을 학습하고 얻어지는 파라미터를 옵티마이저에 전달해서 전달
params_to_update = []
for name, param in resnet18.named_parameters():
    if param.requires_grad == True:
        params_to_update. append (param)#파라미터 학습 결과를 저장
        print("\t", name)
optimizer = optim.Adam(params_to_update) #학습 결과를 옵티마이저에 전달

```

```

fc.weight
fc.bias

```

- 결과로 옵티마이저로 전달되는 파라미터들이 보여준다.

9. 모델 학습

```

#모델 학습
device=torch.device("cuda" if torch.cuda.is_available() else "cpu")
criterion=nn.CrossEntropyLoss() #손실함수 지정
#모델의 정확도와 오차를 반환
train_acc_hist,train_loss_hist=train_model(resnet18,train_loader,criterion,optimizer,device)

```

- 전달되는 파라미터는 (모델, 학습 데이터, 손실 함수, 옵티마이저, 장치)

[테스트 데이터로 모델 정확도 측정하기]

1. 테스트 데이터 호출 및 전처리

```

# 테스트 데이터 호출 및 전처리
test_path='/Users/bluecloud/Documents/대학/유런/data/catanddog/test'

```

```

transform=transforms.Compose([
    transforms.Resize(224),
    transforms.CenterCrop(224) ,
    transforms.ToTensor()
])
test_dataset=torchvision.datasets.ImageFolder(root=test_path,transform=transform)
test_loader=torch.utils.data.DataLoader(
    test_dataset,
    batch_size=32,
    num_workers=1,
    shuffle=True
)
print(len(test_dataset))

```

2. 테스트 데이터 평가를 위한 함수를 생성

```

# 테스트 데이터 평가를 위한 함수를 생성
def eval_model(model,dataloaders,device) :
    since = time.time()
    acc_history = []
    best_acc = 0.0

    saved_models = glob.glob('/Users/bluecloud/Documents/대학/유런/data/catanddog/'+ '*.pth')
    saved_models.sort() #불러온 .pth 파일들을 정렬
    print( 'saved_model' , saved_models)

    for model_path in saved_models:
        print('Loading model', model_path)
        model.load_state_dict(torch.load(model_path))
        model.eval()
        model.to(device)
        running_corrects = 0

        for inputs, labels in dataloaders: # 테스트 반복
            inputs = inputs.to(device)
            labels = labels.to(device)

            with torch.no_grad(): #autograd를 사용하지 않겠다는 의미
                outputs = model(inputs) #데이터를 모델에 적용한 결과를 outputs에 저장
                _,preds = torch.max(outputs.data, 1)
                preds[preds>= 0.5] = 1 # torch.max로 출력된 값이 0.5 보다 크면 올바르게 예측
                preds[preds<0.5] = 0 #torch.max 로 출력된 값이 0.5 보다 작으면 틀리게 예측
                running_corrects += preds.eq(labels.cpu()).int().sum()

        epoch_acc = running_corrects.double() / len(dataloaders.dataset) #테스트 데이터의 정확도 계산
        print('Acc: {:.4f}'.format(epoch_acc))

        if epoch_acc > best_acc:
            best_acc = epoch_acc
            acc_history.append(epoch_acc.item())
            print()

    time_elapsed = time.time()-since
    print('Validation complete in {:.0f}m {:.0f}s'.format(time_elapsed//60,time_elapsed%60))
    print('Best Acc: {:.4f}'.format(best_acc))
    return acc_history #계산된 정확도 반환

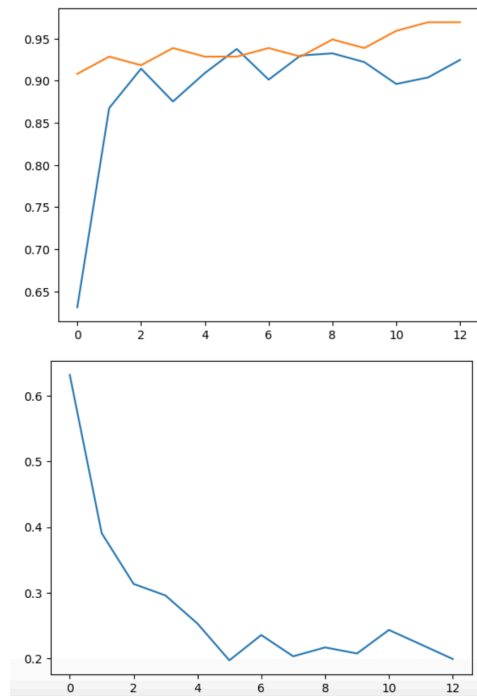
```

- glob : 현재 디렉토리에서 원하는 파일들만 추출하여 가져올 때 사용함 >>.pth 확장자만 갖는 파일 가져옴.
- torch.max : 텐서 배열의 최댓값이 있는 index 를 반환

3. 테스트 데이터를 평가 함수에 적용

```
val_acc_hist = eval_model(resnet18,test_loader,device)
```

4. 시각화 및 오차 정보



5. 예측 이미지 출력 위한 전처리 함수

- tensor.clone() : 기존 텐서의 내용을 복사한 텐서를 생성 + detach() : 기존 텐서에서 기울기가 전파되지 않음 >> 복사한 새로운 텐서를 생성하지만 기울기에 영향을 주진 않음
- clip() : 입력값을 0과 1 사이의 값으로 제한한다

구분	메모리	계산 그래프 상주 유무
tensor.clone()	새롭게 할당	계산 그래프에 계속 상주
tensor.detach()	공유해서 사용	계산 그래프에 상주하지 않음
tensor.clone().detach()	새롭게 할당	계산 그래프에 상주하지 않음

*계산 그래프? 여러 개의 노드와 노드를 연결하는 엣지로 구성되어 계산 과정을 그래프로 나타낸 것 >> 국소적 계산 가능 및 역전파 미분 계산에 편리

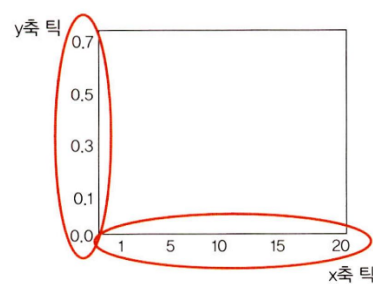
*연쇄 법칙 : 두 개 이상의 함수가 결합된 함수

[테스트 데이터셋으로 개 고양이 분류 잘하나 확인해보기]

1. 개고양이 예측 결과 출력

- add_subplot(2, 10, idx+1, xticks=[], yticks=[])

: 행의 수, 열의 수 인덱스, tick을 삭제



- 예측 결과 : `color=("green" if preds[idx]==labels[idx] else "red" >> 해당 인덱스에 해당하는 이미지의 예측값과 라벨링값이 같으면 초록색, 다르면 빨간색으로 표시`

미세 조정 기법

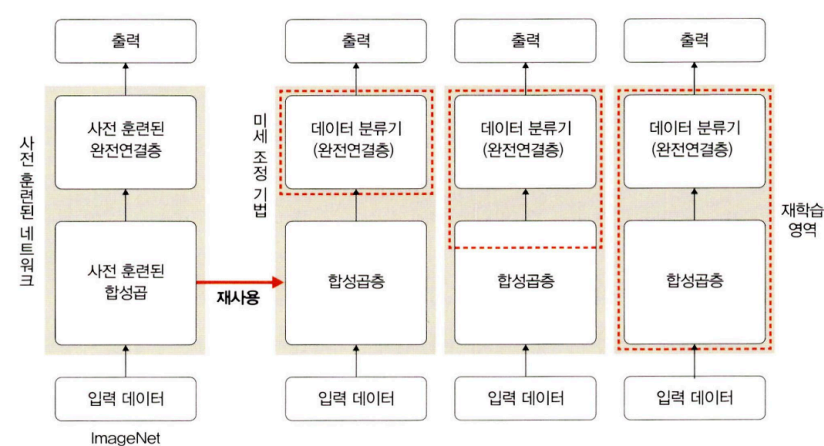
1. 미세 조정 기법?

- 미세 조정 기법이란? 앞선 기법에서 더 나아가 사전 훈련된 모델과 합성곱층, 데이터 분류기이 가중치를 업데이트하며 훈련시키는 방식
- 특성 추출 : 목표 특성이 잘 추출했다는 전제하에 좋은 성능이 나옴 vs 미세 조정 기법 : 잘못 추출되었으면 새로운 이미지 데이터를 사용해 가중치 업데이트해서 다시 추출 가능
- 더 많은 연산량이 필요해 GPU가 권장됨

2. 전략? 데이터셋 크기와 사전 훈련된 모델에 따라 다름

- 크고 유사성 작음 : 전체 재학습
- 크고 유사성 큼 : 합성곱층의 뒷부분(완전 연결층과 가까운 부분)과 데이터 분류기 학습
- 작고 유사성 작음 : 합성곱층의 일부분과 데이터 분류기 학습
- 작고 유사성 큼 : 데이터 분류기만 학습 >> 과적합 이슈때문에

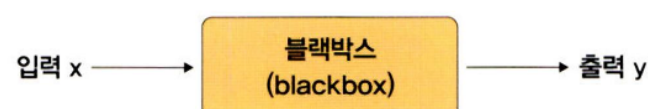
▼ 그림 5-44 미세 조정 기법



>> 파라미터에 큰 변화를 주면 과적합 문제가 발생하므로 정교&미세한 업데이트 필요

▼ 4. 설명 가능한 CNN

설명 가능한 CNN? 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술 >> 처리 과정을 시각화한다거나..



특성 맵 시각화

- 특성 맵 : 입력 이미지 또는 다른 특성 맵처럼 필터를 입력에 적용한 결과
- >> 이를 시각화한다? 입력 특성 감지하는 방법을 이해할 수 있도록 돕는 것

1. 라이브러리 먼저 설치

2. 네트워크 생성 : 13개의 합성곱층 + 두 개의 완전연결층 >> 렐루라는 활성화 함수 사용

```
class XAI(torch.nn.Module):
    def __init__(self, num_classes=2):
        super(XAI, self).__init__()
        self.features = nn.Sequential(
```

```

nn.Conv2d(3, 64, kernel_size=3, bias=False),
nn.BatchNorm2d(64),
nn.ReLU(inplace=True),
nn.Dropout(0.3),
nn.Conv2d(64, 64, kernel_size=3, padding = 1, bias=False),
nn.BatchNorm2d(64),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),

nn.Conv2d(64, 128, kernel_size=3, padding = 1, bias=False),
nn.BatchNorm2d(128),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(128, 128, kernel_size=3, padding = 1, bias=False),
nn.BatchNorm2d(128),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),

nn.Conv2d(128, 256, kernel_size=3, padding = 1, bias=False),
nn.BatchNorm2d(256),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(256, 256, kernel_size=3, padding = 1, bias=False),
nn.BatchNorm2d(256),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(256, 256, kernel_size=3, padding = 1, bias=False),
nn.BatchNorm2d(256),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),

nn.Conv2d(256, 512, kernel_size=3, padding = 1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding = 1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding = 1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),

nn.Conv2d(512, 512, kernel_size=3, padding = 1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding = 1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding = 1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),
)
self.classifier = nn.Sequential(

```



```

        nn.Linear(512, 512, bias=False),
        nn.Dropout(0.5),
        nn.BatchNorm1d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.5),
        nn.Linear(512, num_classes)
    )

```

```

def forward(self, x):
    x = self.features(x)
    x = x.view(-1, 512)
    x = self.classifier(x)
    return F.log_softmax(x)

```

3. 모델 객체화 후 할당

```

model=XAI()
model.to(device)
model.eval()

```

4. 특성 맵 확인 위한 클래스 정의

```

class LayerActivations:
    features=[]
    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn)
    def hook_fn(self, module, input, output):
        output = output
        self.features = output.to(device).detach().numpy()
    def remove(self):
        self.hook.remove()

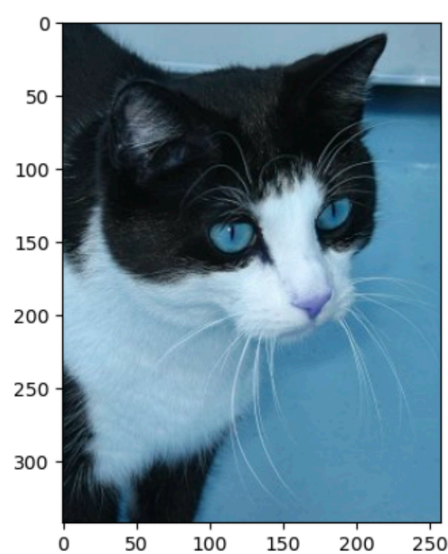
```

5. 이미지 호출

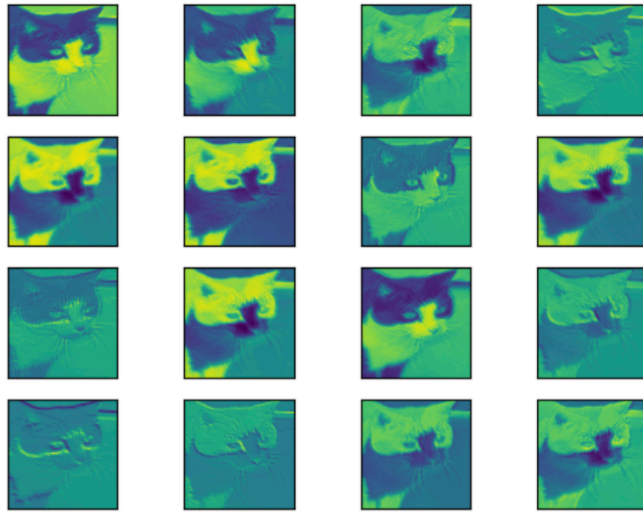
```

img=cv2.imread("/Users/bluecloud/Documents/대학/유런/data/catanddog/test/Cat/8113.jpg")
plt.imshow(img)
img = cv2.resize(img, (100, 100), interpolation=cv2.INTER_LINEAR)
img = ToTensor()(img).unsqueeze(0)
print(img.shape)

```

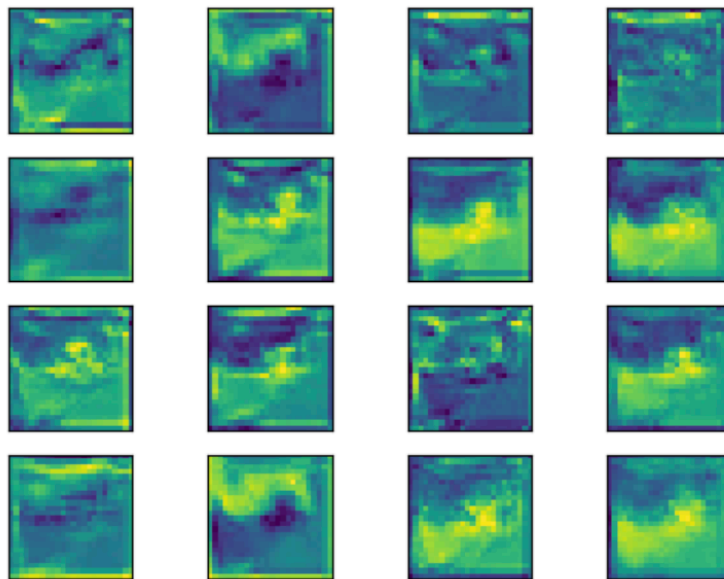


6. 특성 맵 확인 : 첫 번째 Conv2d 계층에서 특성 맵에 대한 출력 결과



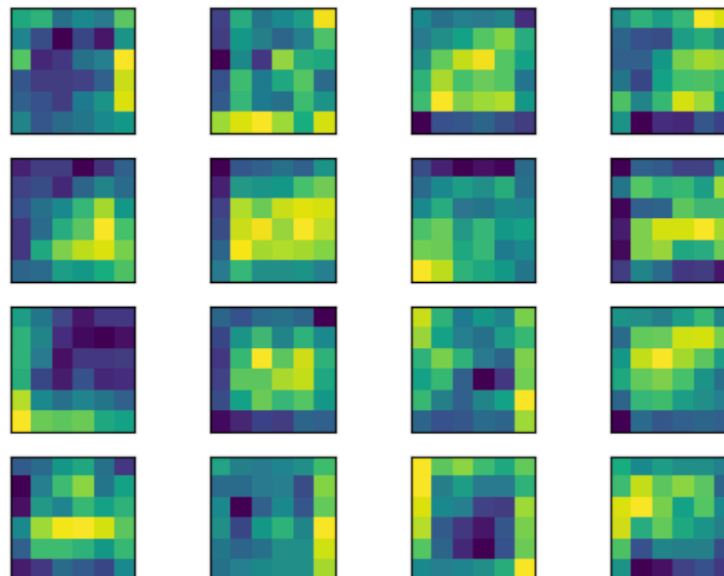
7. 20번째 계층에 대한 특성 맵 살펴보기

8. 시각화



9. 40번째 계층에 대한 특성 맵 살펴보기

10. 시각화



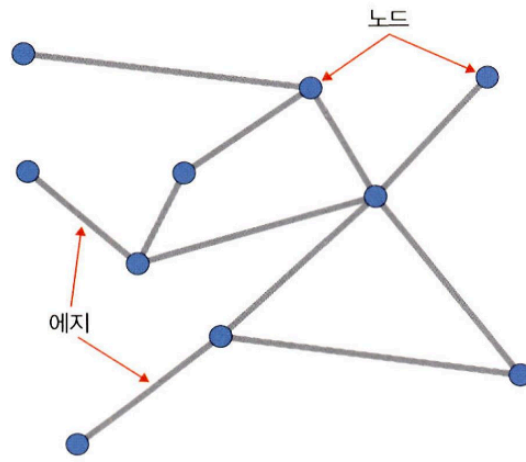
>> 출력층에 가까울수록 원래 형태를 찾아볼 수 없다. & 이미지 특징들만 전달된다.

▼ 5. 그래프 합성곱 네트워크

그래프 합성곱 네트워크란? 그래프 데이터를 위한 신경망

그래프란

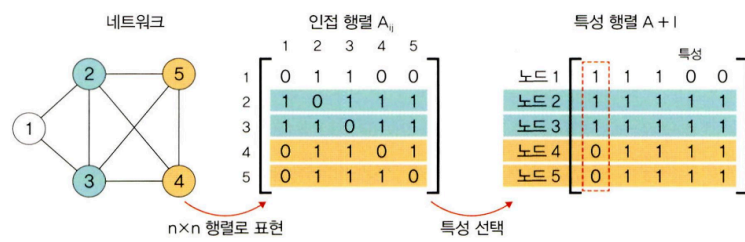
- 그래프란? 방향성이 있거나 없는 엣지로 연결된 노드의 집합



- 노드 : 원소 / 엣지 : 결합 방법

그래프 신경망(GNN)

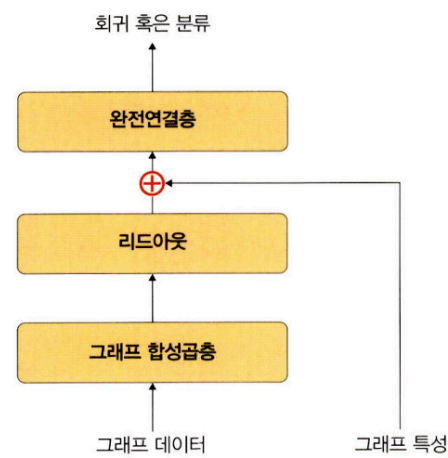
- GNN : 그래프 구조에서 사용하는 신경망



- 1단계 : 인접 행렬 >> i와 j의 관련성을 (i,j)값으로 채워진다.
- 2단계 : 특성 행렬 >> 단위 행렬을 적용한다. 즉 각 입력 데이터에서 이용할 특성을 선택해 그래프 특성이 추출된다.

그래프 합성곱 네트워크(CGN)

- 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘



↳ 리드아웃 : 특성 행렬을 하나의 벡터로 변환하는 함수 >> 전체 노드의 특성 벡터에 대한 평균을 구하고 그래프 전체를 표현하는 하나의 벡터를 생성

↳ 그래프 합성곱층 : 적용하면 그래프 형태의 데이터가 행렬 형태로 변환되어 딥러닝 알고리즘에 적용 가능

- 적용 예시 : SNS 에서 관계 네트워크, 학술 연구에서 인용 네트워크, 3D Mesh